

2024-2 학기

Cloud Computing Term Project 1: Local to Cloud Migration

학번: 2019312014

이름: 박병준

학과: 컴퓨터교육과

과목: 클라우드컴퓨팅개론

담당 교수: 최윤석 교수님

1. 프로젝트 소개

로컬 컴퓨터에서 원활한 구동이 가능한 프로젝트를 AWS 클라우드 환경으로 migration 해서 배포하는 과정을 수행하고자 한다. 여기서는 기존 소스코드를 활용한 간단한 웹 페이지 제작 프로그램을 클라우드 배포 대상으로 선정했다. 해당 프로젝트는 공방 소개 웹사이트를 주제로 제작되었고, 11 개의 html 파일, 12 개의 css 파일, 3 개의 js 파일, 12 개의 image 파일로 구성되어 있으며, 소스코드는 다음 github 주소에 배포되어 있다.

https://github.com/bjpark0925/young_craft_workshop_website

웹사이트는 다음 주소를 통해 접근할 수 있다.

https://bjpark0925.github.io/young_craft_workshop_website/main.html

과제는 다음과 같이 ①Dockerfile 작성, ②Docker 이미지 생성 및 테스트, ③AWS EC2 인스턴스 생성, ④EC2 에서 Docker 설치, ⑤Docker 이미지 실행 및 웹 페이지 확인의 5 가지 순서로 진행했다. 첫째로, 프로젝트를 Dockerize 하기 위해 프로젝트 파일을 하나의 컨테이너로 패키징할 수 있는 Dockerfile 을 작성했다. 프로젝트의 루트 디렉토리에 vi Dockerfile 명령어로 Dockerfile 을

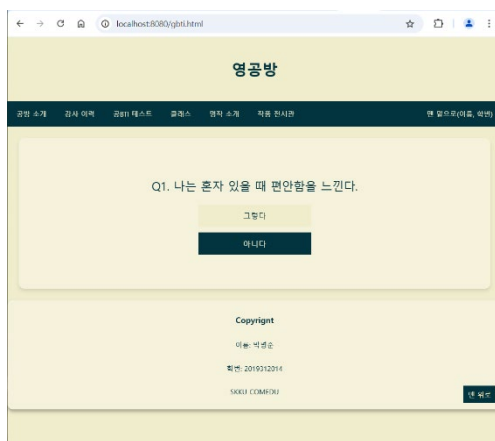
생성한 뒤, 다음과 같은 내용을 Dockerfile 에 작성했다.

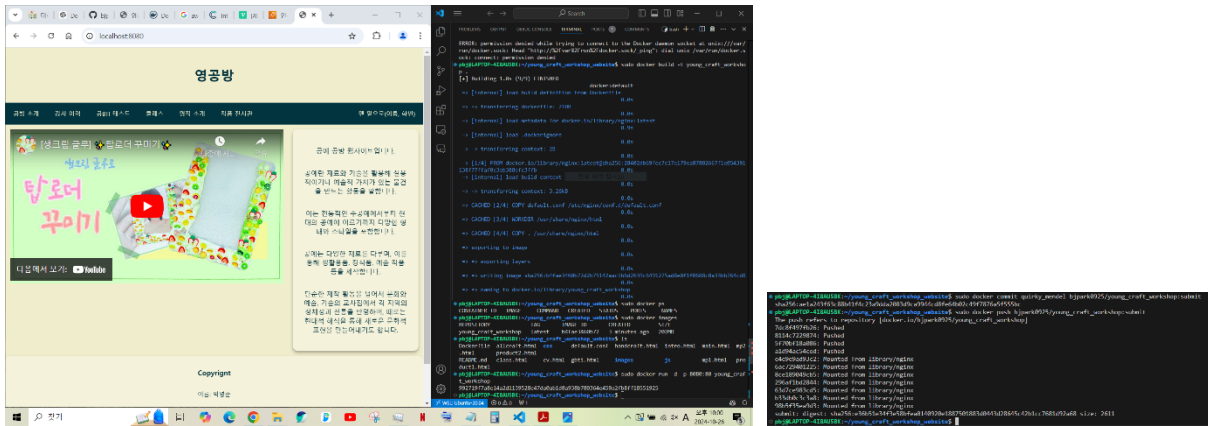
```
1 FROM nginx:latest
2 COPY default.conf /etc/nginx/conf.d/default.conf
3 WORKDIR /usr/share/nginx/html
4 COPY . /usr/share/nginx/html
5 EXPOSE 80
6 CMD ["nginx", "-g", "daemon off;"]
~
~
~
~
~
"Dockerfile" 6L, 171C
```

이를 통해 컨테이너가 실행될 때 Nginx 웹 서버를 사용하여 디렉토리 내 파일들이 정적으로 서빙될 수 있다. 여기서 추가적으로 고려해야 할 점이 있는데, Nginx 웹 서버는 기본적으로 웹 브라우저에서 접속할 때 index.html 파일을 첫 페이지로 보여주므로, main.html 파일을 첫 페이지로 설계한 내 프로젝트에 맞추기 위해서는 추가적인 작업이 필요했다. 따라서 루트 디렉토리에 default.conf 파일을 만들고 다음과 같은 내용을 작성했다.

```
1 server {
2     listen 80;
3     server_name localhost;
4
5     location / {
6         root    /usr/share/nginx/html;
7         index   main.html index.html;
8     }
9 }
10
```

둘째로, 로컬 환경에서 Docker 이미지를 생성하고, 웹 페이지 동작을 테스트했다. 이를 위해 docker build 와 docker run 명령어를 사용했다. 브라우저에서 <http://localhost:8080>으로 접속해 웹 페이지가 올바르게 동작하는지 확인했다.



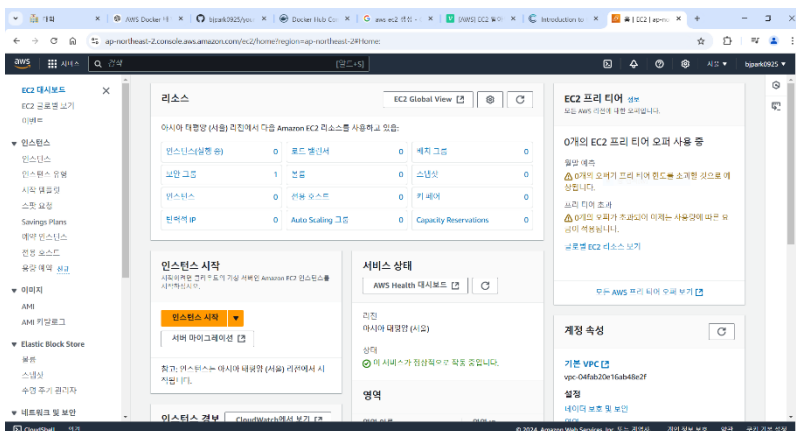


실행에 쓰인 Docker 이미지는 EC2 에서 clone 해 가져오기 위해 dockerhub 에 push 했다. 도커 이미지가 저장된 dockerhub 주소는 다음과 같다.

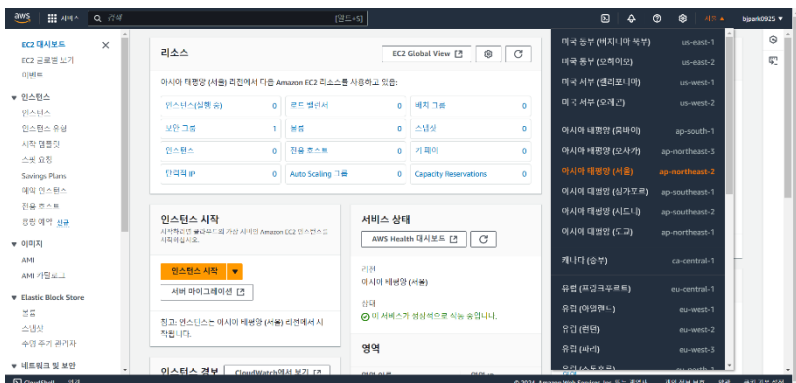
https://hub.docker.com/repository/docker/bjpark0925/young_craft_workshop/general

셋째로, AWS 관리 콘솔에 로그인하고 EC2 서비스로 이동해 새로운 인스턴스를 생성했다.

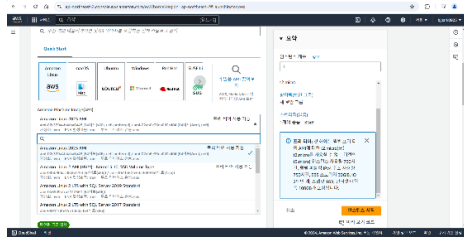
EC2 대시보드 모습



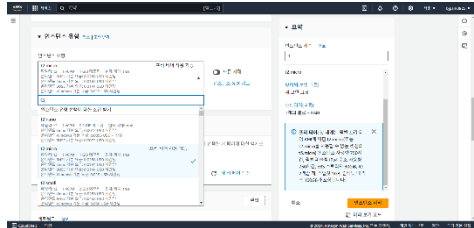
클라우드 서버의 지역 선택도 가능했다. 주로 서비스되는 지역의 클라우드 서버를 선택하면 서비스에 지연되는 시간이 줄어드는 효과가 있다.



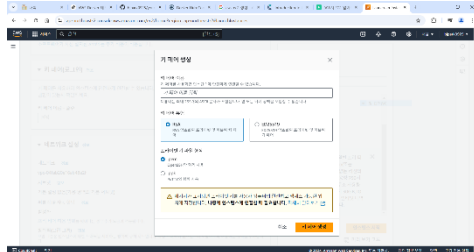
Amazon Machine Image 선택



인스턴스 유형 선택 - t2.micro



키 페어 생성 - 다운로드된 키 페어 파일이 있어야만 인스턴스 연결이 가능하다.

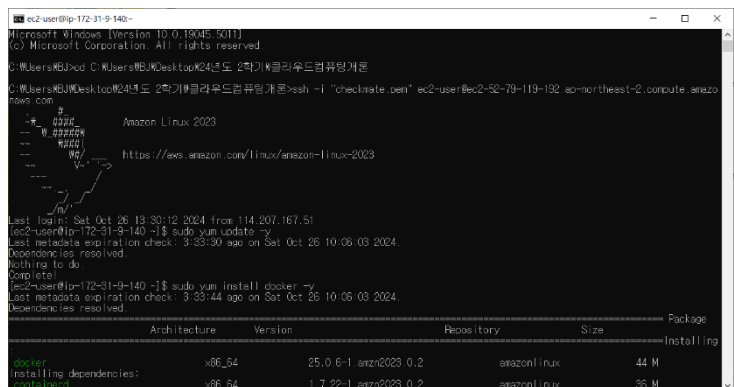


인바운드 규칙 설정 - 기본적으로 SSH 22 번 포트만 설정되어 있으며, 웹사이트 구동을 위해

HTTP 80 번 포트도 열어주었다. 이외에도 HTTPS 를 비롯해 다양한 네트워크 설정이 가능하다.



넷째로, EC2 에 접속하기 위해 SSH 접속 방법을 알아내어 터미널 창을 통해 실행했다.



EC2 에서 Docker 를 설치하고, dockerhub 에 저장돼 있는 Docker 이미지를 pull(download)했다.

```
[ec2-user@ip-172-31-9-140 ~]$ sudo service docker start
Redirecting to /bin/systemctl start docker.service
[ec2-user@ip-172-31-9-140 ~]$ sudo usermod -aG docker ec2-user
[ec2-user@ip-172-31-9-140 ~]$ sudo docker image pull bjpark0925/young_craft_workshop:submit
submit: Pulling from bjpark0925/young_craft_workshop
a480a496ba95: Pull complete
f3ace1b8ce45: Pull complete
11d6fdd0e8a7: Pull complete
f1091da6fd5c: Pull complete
40eea07b53d8: Pull complete
6476794e50f4: Pull complete
70850b3ec6b2: Pull complete
51d731db9542: Pull complete
4f4fb700ef54: Pull complete
19f925a68ccd: Pull complete
32f44445c8f6: Pull complete
Digest: sha256:e36b51e34f3e58bfea0140920e1887501883d0443d28645c42b1cc7681d92a68
Status: Downloaded newer image for bjpark0925/young_craft_workshop:submit
docker.io/bjpark0925/young_craft_workshop:submit
```

마지막으로, EC2 에서 Docker 이미지를 실행시켜 웹 페이지가 정상적으로 동작하는지 확인했다.

```
[ec2-user@ip-172-31-9-140 ~]$ sudo docker images
REPOSITORY          TAG                 IMAGE ID            CREATED             SIZE
bjpark0925/young_craft_workshop  submit             ae1a243f63c8       42 minutes ago     202MB
[ec2-user@ip-172-31-9-140 ~]$ sudo docker ps
CONTAINER ID        IMAGE               COMMAND             STATUS             PORTS             NAMES
[ec2-user@ip-172-31-9-140 ~]$ sudo docker run -d -p 80:80 bjpark0925/young_craft_workshop:submit
770c6ded89bd7cfa0a9bad0fa064847538b39a1a4147b47eb8616113cf799030
[ec2-user@ip-172-31-9-140 ~]$
```

EC2 인스턴스의 퍼블릭 IP 를 브라우저에서 입력하는 식으로 웹 페이지에 접근했다. 퍼블릭 IP 는 AWS 관리 콘솔의 EC2 인스턴스 세부 정보에서 확인할 수 있었다. 다음은 클라우드 환경에서 동작하는 웹 사이트를 캡처한 화면이다.



2. 로컬 개발과 클라우드 배포 간 장단점 비교

로컬 개발의 장점은 다음과 같다. 첫째, 로컬 환경에서 코드를 작성하고 곧바로 실행할 수 있기 때문에 개발 및 테스트 속도가 빠르다. 둘째, 클라우드 사용 비용이 들지 않는다. 셋째, 인터넷이 없어도 오프라인으로 작업할 수 있다. 반면, 로컬 개발의 단점은 다음과 같다. 첫째, 로컬 환경과 실제 배포 환경이 다를 수 있고, 운영 환경에서 문제가 발생할 수 있다. 둘째, 로컬 컴퓨터의

성능에 따라 대규모 애플리케이션을 개발하고 업데이트하는 데 한계가 있다. 셋째, 여러 개발자가 협업할 때, 로컬 환경에서의 충돌이나 버전 관리 문제에 봉착할 수 있다.

이에 반해 클라우드 배포는 로컬 개발의 장단점과 상충되는 특징이 있다. 클라우드 배포의 장점으로서는 첫째, 배포 환경과 운영 환경을 동일하게 구성할 수 있기 때문에 운영 환경의 불일치로 인한 문제를 해결할 수 있다. 둘째, AWS 와 같은 클라우드 환경은 자동으로 자원을 확장할 수 있어 트래픽 증가나 성능 요구에 유연하게 대응할 수 있다. 셋째, 클라우드에 배포된 애플리케이션은 전 세계 어디서든 접속 가능하므로, 실제 사용자 테스트나 협업에 유리하다. 뿐만 아니라, 개발자가 신경 쓰지 않아도 자동 백업 및 복구 기능이 지원되어 안정성이 높다. 대시보드를 통해 모니터링도 쉽게 할 수 있다.



반면, 클라우드 배포의 단점은 다음과 같다. 첫째, 클라우드 사용량에 비례하여 비용이 발생하는 특징으로 인해 과도한 트래픽이 발생하면 예상치 못한 비용이 발생할 수 있다. 둘째, 클라우드에서 작업할 때는 인터넷이 필수적이므로 로컬 환경과 달리 오프라인에서 작업할 수 없다.

3. 서버 규모에 따른 클라우드 마이그레이션 분석

소규모 서버는 트래픽이 적고, 데이터의 양도 많지 않으므로 마이그레이션이 비교적 간단하다. AWS EC2 인스턴스를 사용하여 기존 서버의 환경을 클라우드로 그대로 옮기거나, AWS Lambda 와 같이 서버리스 컴퓨팅을 사용하여 비용 절감을 꾀할 수 있다.

여러 어플리케이션이나 마이크로서비스를 제공하는 중규모 서버의 경우, 마이그레이션 시 각 서비스가 독립적으로 확장되도록 설계하고, 각 서비스를 Docker 로 컨테이너화해 확장성을 갖추도록 할 수 있다.

대규모 서버 환경은 데이터 복잡성이 높고 리소스 요구사항이 많아 마이그레이션이 복잡하고 시간이 오래 소요될 수 있다. 마이그레이션 시 기존 애플리케이션 전체를 클라우드 환경에 최적화되도록 수정 및 개선하거나, 일부만 온프레미스 환경에 남기고 중요한 서비스만 클라우드로 이전하는 하이브리드 아키텍처를 구축할 수 있다.

4. 보안/백업/유지보수 방법

클라우드 환경에서 보안을 강화하기 위한 방법으로 SSH 나 HTTPS 같은 프로토콜을 사용할 수 있다. 암호화를 통해 데이터를 안전하게 전송할 수 있다는 장점이 있다. SSH 의 경우, 네트워크 상에서 원격 시스템에 안전하게 접속하고 명령을 실행할 수 있게 해주는 암호화 통신 프로토콜이다. 주로 서버에 원격으로 접속하거나 파일을 전송할 때 사용된다. 주로 공개키/비밀키 쌍을 이용한 인증 방식을 사용한다. AWS EC2 인스턴스에 SSH 키 페어를 사용하여 접속하는 것이 그 예이다. HTTPS 의 경우, 웹 브라우저와 서버 간 통신 규약인 HTTP 프로토콜에 보안 계층을 추가한 것으로, SSL/TLS 인증서를 통해 서버의 신원을 검증하는 특징이 있다. 웹 상에서 민감한 정보가 오고 가는 경우 사용될 수 있다.

백업을 위한 방법으로는, 자동 스냅샷 설정을 통해 특정 주기마다 데이터베이스 상태를 자동 저장하는 방법이 있다. 이외에도, 데이터를 분산 저장해 예기치 못한 사고로 인한 데이터 손실을 예방하고, 서드파티 백업 도구의 힘을 빌려 데이터 백업을 자동화할 수 있다.

유지보수를 위한 방법으로는, 트래픽 증가나 감소에 따라 리소스를 자동으로 조절하는 오토스케일링을 설정하여 유지보수를 용이하게 할 수 있다. 또한, 하나의 인스턴스에만 과부하가 걸리지 않도록 로드 밸런서를 통해 여러 인스턴스로 트래픽이 분산되도록 할 수 있다. 그리고 리소스 상태를 모니터링할 수 있는 대시보드와 문제 발생 시 빠른 대응이 가능한 알림 시스템이 도움이 될 수 있다. 이외에도 클라우드의 보안 패치와 시스템 업데이트를 자동화하거나, 테스트 환경을 운영 환경과 논리적으로 격리시켜 유지 보수하는 방법이 있다.

5. 결과 요약

로컬 환경에서의 개발과 클라우드 배포 간 다양한 장단점을 비교 분석할 수 있었다. 그 결과 기업 운영의 관점에서 볼 때, 개발 초기에는 로컬 환경에서 빠르게 개발하는 것이 효과적이고, 운영 환경에 가깝게 테스트하거나 대규모 애플리케이션을 배포할 때는 클라우드로 마이그레이션해 배포하는 것이 효과적이라는 것을 밝혔다. 그리고 소규모 서버는 기존 애플리케이션을 거의 변경 없이 그대로 클라우드로 마이그레이션하는 방식을 제시했고, 이에 반해 대규모 서버는 클라우드 마이그레이션 시 코드나 애플리케이션 구조를 변경하여 클라우드 내 기능을 최대한 활용할 수 있도록 재설계하는 방식을 제시했다. 마지막으로, 클라우드 환경에서 데이터를 안전하게 주고받는 프로토콜을 통해 보안 문제를 해결하고, 데이터 복구 가능성을 높이는 백업 방식을 사용하고, 다양한 유지보수 방법을 통해 시스템의 안정성과 성능을 보장할 수 있다고 했다. 클라우드 환경은 장점이 명확하기에, 클라우드 사용 시 발생할 문제점들을 해결하는 전략들을 앞서 언급한 바와 같이 실천한다면 소프트웨어 기술 발전에 도움을 줄 것이라고 생각한다.